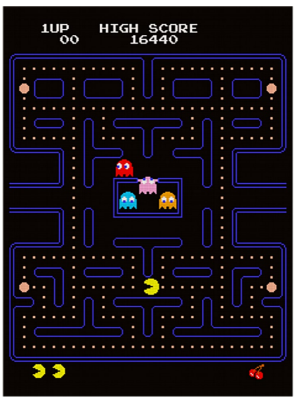
Sandra Greiner
greiner@imada.sdu.dk

Group Project

Project description



The goal of this project is to design and develop a simple game in the style of the 1980 arcade classic Pac-Man using Java. Although the evaluation will focus on **object-oriented design and programming**, the topic offers the opportunity to apply knowledge and skills gained from other courses in this semester, especially DM577 Introduction to Artificial Intelligence and DM578 Algorithms and Data Structures.

Requirements

Your implementation of a Pac-Man-like game should satisfy the criteria below:

- A game score and life counter is tracked and visible to the player at all times.
- The game has four *states*: `Normal`, `Immune`, `Power`, and `Finished`.
  The game begins in the state `Normal` with a life counter of two.
- When the game starts, Pac-Man is in a maze filled with small pellets, four large pellets, and four ghosts (see screenshot from original Pac-Man game for reference). The objective of the game is for Pac-Man to eat all pellets in the maze while avoiding the ghosts.
- Pac-Man moves continuously in the direction it is facing at all times. This direction is controllable through keyboard presses, for example using the arrow keys. When Pac-Man collides with a wall, it remains stationary until its direction is changed to face away from a wall.
- Like Pac-Man, Ghosts move continuously within the maze at a reasonable speed (e.g., they cannot cross walls or move discontinuously from one place to another).
- The behavior of the ghosts depends on the game state.
  - In the state `Normal`, the ghosts move such as to pursue Pac-Man.
  - In the state `Power`, the ghosts move such as to avoid Pac-Man.
- When Pac-Man collides with a *small* pellet, the pellet disappears.
- When Pac-Man collides with a *power* pellet, the pellet disappears, and the game state changes to the state `Power` for a number of seconds (e.g., 10-20 seconds) before reverting back to the state `Normal`.
  It should be visually indicated when the game is in the state `Power`, e.g., by coloring the ghosts gray.
- When Pac-Man collides with a ghost, what happens depends on the state of the game.
  - If the game is in the state `Normal`, the life counter decreases by one.
    - If at this point the life counter is at 0, the game ends and changes to the state `Finished`.
    - If at this point the life counter is larger than 0, the game changes to the state `Immune` for a short period of time (e.g., 1-2 seconds) before reverting back to the state `Normal`.
  - If the game is in the state `Immune`, nothing happens.
  - If the game is in the state `Power`, the ghost disappears.
- The game keeps track of the score, and the following game actions increase the score:
  - Pac-Man colliding with a small pellet (in any state).
  - Pac-Man colliding with a power pellet (in any state).
  - Pac-Man colliding with a ghost while in the state `Power`.

  Come up with a point scheme yourself – it can be as simple or as complex as you prefer.

- In the state `Finished`, the game displays the latest score, and offers the option to start a new game (which would reset the score).

Please note:
The above description contains the *minimal* requirements of the project. You are free to implement additional mechanisms or rules (e.g., different ghosts may have different behaviors, there may be additional power ups or ways of getting additional lives, etc.), but do focus on the core requirements first.

It is possible to implement a Pac-Man game with Java and JavaFX in an ad-hoc way without considering any object-oriented design principle, design patterns, etc. However, this will most likely result in code that is difficult to read and maintain and that fares poorly on the course objectives.

For sprites, you are welcome to find ones online, or even design your own (just remember that this is not part of the evaluation, this is not a course about game design).

Project management and reviews

This is project is not trivial and takes several weeks. This requires some planning and careful time management. To help you keeping on track and providing early feedback, we will structure the process along the following deadlines:

- **08./09.04.** peer-review in lab classes; shortly present your current ideas mainly on design (ca. 5-10minutes/team, details will follow on Itslearning before Easter)
- **29./30.04.** peer-review in lab classes; shortly present the current state of your ideas and implementation (specifically where you still have questions)
- **17.05.** hand-in of implementation
- **19.05.** hand-in of report

Hand-in

On Itslearning, you must hand-in

- a zip-file that contains a folder named `project`. The project-folder should contain your implementation, and a `README` explaining how to run your source code
- a PDF document named `report.pdf` containing your report

Source Code

Your code must be documented and adhere to the common coding conventions and rules of Java. Using Maven for your project is encouraged but not required; should you decide to use a solution other than Maven you should first discuss it with the course teachers.

You can use packages and features from the Java standard library and JavaFX, and source code from the course material. Use of third-party packages and libraries needs to be approved by the course teacher on a case-by-case bases.

Note on usage of Generative AI: Refrain from using AI to implement larger code components in your code base (auto-complete of smaller code snippets is OK). The purpose of the project is to gain experience with the process of refining and translating requirements specifications into concrete OO design and implementation in Java. AI short-circuits this process, which is detrimental to learning.

Some tips on how develop the source code

- Structure your code into classes that follow the object-oriented design principles we have discussed in the lessons. Organize coherent classes into packages.
- Feel free to use design patterns, but remember that design patterns are not a checklist. Use them when/if it makes sense.

- Your implementation should reflect the UML diagram in your report (see below). It is common practice to design the diagram before you start implementing, and to refine the diagram as you go along when/if your implementation evolves.
- We expect you to use JavaFX. You are also welcome to use standard libraries from Java's JDK, as well as libraries for testing, such as JUnit. Apart from these libraries, we expect that all other code is your own. If there are libraries you want to use in addition to these, please write to the course coordinator, Sandra Greiner (email show on first page).
- Decide how you ensure the quality (test) of your code. Follow this method throughout the project, unless there are good reasons to change the testing strategy. Justify your choice of method and, if applicable, your reasons for changing strategy.

Report

The group report serves one main purposes:

> It gives the assessors an overview and insight into your project, process, and considerations.

The sections in the template below help you structure your project report. We recommend that you write the report on an ongoing basis throughout the project.

Formalities

In your report, you must describe your implementation explaining and motivating its structure and functionality. Your report must be written in English or Danish and be delivered as a single PDF file printable in black and white (avoid dark backgrounds for code listings) and with at most 15 pages. You may include additional material as an appendix which does not count towards the page limit (but it is up to the teachers whether to consider it or not). The report must state the name of the group and list its members (full name and SDU email, where applicable). You are provided with a group report Latex-template, which you are free to use.

Note on usage of Generative AI : You may use Generative AI for spell checking and fixing grammar issues. However, if you happen to use Generative AI, to write large parts of your report, it must be declared in the report. Be aware, that this may **count negatively towards your grade**, as it is expected from you that you write the report by yourselves.

Project report template structure

1. **Introduction**
   - Assume that the reader knows the requirements specification and refrain from repeating unnecessary details from it.
   - One purpose of this section is to document any deviations from the requirements specification.
   - For example, have you made simplifying assumptions regarding the specification? Or interpreted any ambiguities in the specification? Added new requirements? Describe which ones and how you may have interpreted the specification.
   - Keep it brief, especially if you have only few deviations.
2. **Design**
   - Include a UML diagram and a description of your software design.
   - Provide a brief description of your diagram:
     - What are the different parts?
     - Have you used design patterns in your design? If so, where in the diagram are they found? It is not a requirement to use design patterns, but it can be a good idea for certain parts.
   - Document design decisions where you have used SOLID, DRY, or other OO principles.

- If you have refined your design during the project, provide a brief description of the changes you have made and why.

3. **Implementation**
   The purpose of this section of the report is to offer readers an overview of the interesting implementation details that are worth looking into in your code base, as well as the necessary details for running your code.

   - Specify which version of Java you used to test and compile your code, and include brief instructions on how to compile and run the code.
   - Provide a high-level description of interesting implementation aspects. For example, aspects that you have spent a lot of time or energy on.
   - These could be more advanced aspects such as how you handle game actions, animation, moving of the ghost, etc.
   - Keep the description general. We can read your code for the details.

4. **Quality assurance**
   This section describes how you tested that your code meets the requirements specification. For example, did you use unit tests? Manual tests?
   Would you have taken a different approach to quality assurance if you had to implement the project again from scratch?

5. **Project Management**
   Here you should present how you organized the team work and project process.

   - Did you work in phases during the project?
   - How did the collaboration go, and how did you ensure equal participation?
   - Did you use technical tools to make collaboration easier across machines?
   - Did you use AI to support the development of your project? If so, how?

6. **Discussion**
   Here you should reflect on how the project went.

   - Would you do anything differently if you had to implement the project from scratch?
   - Were there parts of the project description that you did not get to? If so, how are these parts compatible with your design? Would you make changes to accommodate changes?

Getting started

It is always good to start with tutorials if new programming technology should be used and adapted to specific needs. A tutorial on how to develop basic games in JavaFX is available here:

https://code.tutsplus.com/introduction-to-javafx-for-game-development–cms-23835t

For sprites, feel free to find some online, or even design them on your own (but don't spend too much time on it).